UJIAN AKHIR SEMESTER
DATA MINING & MACHINE LEARNING

Laporan Interpretasi
Pemodelan Prediksi Chronic Kidney Disease (CKD)

LASSO Logistic Regression, Decision Tree, dan Neural Network

Mata Kuliah	:	Data Mining and Machine Learning
Dosen	:	Sigit A. Saputro, S.KM., M.Kes. Ph.D. (D.Sc.H)
Nama	:	Mohammad Maliki Rafli
NIM	:	291251025
Program Studi	:	S2 Kesehatan Masyarakat
Peminatan	:	Biostatistika dan Data Sains Kesehatan
Software	:	Jupyter Notebook dengan Python

PROGRAM STUDI S2 KESEHATAN MASYARAKAT
PEMINATAN BIOSTATISTIKA DAN DATA SAINS KESEHATAN
2026

DAFTAR ISI

I. PENDAHULUAN

1.1 Latar Belakang Analisis

Chronic Kidney Disease (CKD) merupakan kondisi klinis yang dapat dikenali melalui kombinasi indikator fungsi ginjal, parameter darah, dan informasi komorbid. Dalam konteks tugas Data Mining & Machine Learning, dataset CKD digunakan sebagai studi kasus klasifikasi biner untuk membedakan pasien CKD dan Not CKD berdasarkan prediktor klinis dan laboratorium.

Pemodelan prediktif analisis ini tidak hanya menilai apakah model mampu mengklasifikasikan outcome secara akurat, tetapi juga menilai aspek development, internal validation, diskriminasi, kalibrasi, dan interpretabilitas. Oleh karena itu, laporan ini membandingkan tiga algoritma, yaitu LASSO Logistic Regression, Decision Tree, dan Neural Network.

II. DATASET DAN PREPROCESSING DATA

2.1 Deskripsi Dataset dan Target Prediksi

Dataset yang digunakan adalah dataset Chronic Kidney Disease dengan 400 observasi dan 26 kolom awal. Target prediksi adalah status CKD pada variabel classification. Setelah proses cleaning, target dikodekan menjadi CKD = 1 dan Not CKD = 0. Kolom id tidak digunakan sebagai prediktor karena hanya berfungsi sebagai identitas baris, bukan informasi klinis.

Tabel 1. Distribusi target prediksi

Kelas	Jumlah
CKD	250
Not CKD	150

Distribusi target menunjukkan bahwa kelas CKD lebih banyak dibandingkan Not CKD. Proporsi ini masih dapat dikelola dengan stratified split agar komposisi kelas pada development set dan internal validation set tetap relatif seimbang.

Tabel 2. Sepuluh variabel dengan missing value tertinggi

Variabel	Missing n	Missing %
rbc	152	38.0
rc	131	32.8
wc	106	26.5
pot	88	22.0
sod	87	21.8
pcv	71	17.8
pc	65	16.2
hemo	52	13.0
su	49	12.2
sg	47	11.8

Variabel dengan missing value tertinggi adalah rbc, rc, wc, pot, dan sod. Kondisi ini perlu ditangani secara sistematis agar setiap algoritma memperoleh input yang konsisten dan tidak menghasilkan bias teknis akibat nilai kosong yang tidak diproses.

2.2 Tahapan Preprocessing

Preprocessing dilakukan untuk memastikan data dapat digunakan secara konsisten pada seluruh algoritma. Nama kolom dibuat seragam, nilai kategorikal dibersihkan dari spasi dan tab, serta nilai seperti tanda tanya, string kosong, dan nan dikonversi menjadi missing value.

Variabel numerik dikonversi ke format numerik, sedangkan variabel kategorikal dipertahankan sebagai kategori. Missing value numerik diimputasi menggunakan median, sementara missing value kategorikal diimputasi menggunakan modus. Variabel kategorikal kemudian diubah menjadi dummy variable melalui one-hot encoding.

Standardisasi digunakan pada LASSO Logistic Regression dan Neural Network karena kedua model sensitif terhadap skala prediktor. Decision Tree tidak memerlukan standardisasi karena proses pemisahan node berbasis aturan cut-off pada masing-masing variabel.

Tabel 3. Pembagian development set dan internal validation set

Set	n	CKD	Not CKD	Proporsi CKD
Development	300	188	112	62.7%
Internal validation	100	62	38	62.0%

Data dibagi menjadi development set sebesar 75% dan internal validation set sebesar 25% dengan teknik stratified split. Pembagian stratified dipilih agar proporsi CKD dan Not CKD tetap seimbang pada data pengembangan dan data validasi internal.

III. MODEL DEVELOPMENT

3.1 Strategi Pengembangan Model

Model dikembangkan menggunakan stratified 3-fold cross-validation pada development set. Skor utama yang digunakan saat tuning adalah AUC karena outcome berbentuk klasifikasi biner dan AUC mampu menilai kemampuan diskriminasi model pada berbagai threshold.

Setelah hyperparameter terbaik diperoleh, model dilatih kembali pada seluruh development set. Model final kemudian dievaluasi pada internal validation set dengan threshold 0,5 dan CKD sebagai kelas positif.

Tabel 4. Hasil cross-validation pada development set

Model	Best params	Mean CV AUC	Mean train AUC
LASSO Logistic Regression	{'model__C': 1.0}	0.999	1.000
Decision Tree	{'model__ccp_alpha': 0.0, 'model__criterion': 'gini', 'model__max_depth': 2, 'model__min_samples_leaf': 5}	0.982	0.988
Neural Network	{'model__activation': 'relu', 'model__alpha': 0.001, 'model__hidden_layer_sizes': (16, 8)}	0.991	0.990

Hasil cross-validation menunjukkan bahwa ketiga model memiliki kemampuan diskriminasi yang tinggi pada development set. LASSO Logistic Regression memperoleh

Mean CV AUC 0,999, Neural Network 0,991, dan Decision Tree 0,982. Selisih performa tersebut menjadi salah satu dasar awal untuk menilai stabilitas model sebelum dievaluasi pada internal validation set.

IV. EVALUASI MODEL PADA INTERNAL VALIDATION SET

4.1 Metrik Evaluasi

Evaluasi model menggunakan threshold 0,5 dengan CKD sebagai kelas positif. Metrik yang dilaporkan meliputi confusion matrix, accuracy, sensitivity, specificity, precision, F1-score, AUC, dan Brier score. AUC menunjukkan kemampuan diskriminasi, sedangkan Brier score menunjukkan kualitas probabilitas prediksi; semakin kecil Brier score, semakin baik kualitas prediksi probabilistik model.

4.2 Hasil Evaluasi Internal Validation

Tabel 5. Evaluasi model pada internal validation set

Model	TN	FP	FN	TP	Accuracy	Sensitivity	Specificity	Precision	F1	AUC	Brier
LASSO Logistic Regression	38	0	0	62	1.000	1.000	1.000	1.000	1.000	1.000	0.0016
Neural Network	38	0	2	60	0.980	0.968	1.000	1.000	0.984	1.000	0.0665
Decision Tree	37	1	0	62	0.990	1.000	0.974	0.984	0.992	0.998	0.0124

Tabel 6. Interval kepercayaan bootstrap untuk AUC dan accuracy

Model	AUC 95% CI low	AUC 95% CI high	Accuracy 95% CI low	Accuracy 95% CI high
LASSO Logistic Regression	1.000	1.000	1.000	1.000
Decision Tree	0.994	1.000	0.970	1.000
Neural Network	1.000	1.000	0.950	1.000

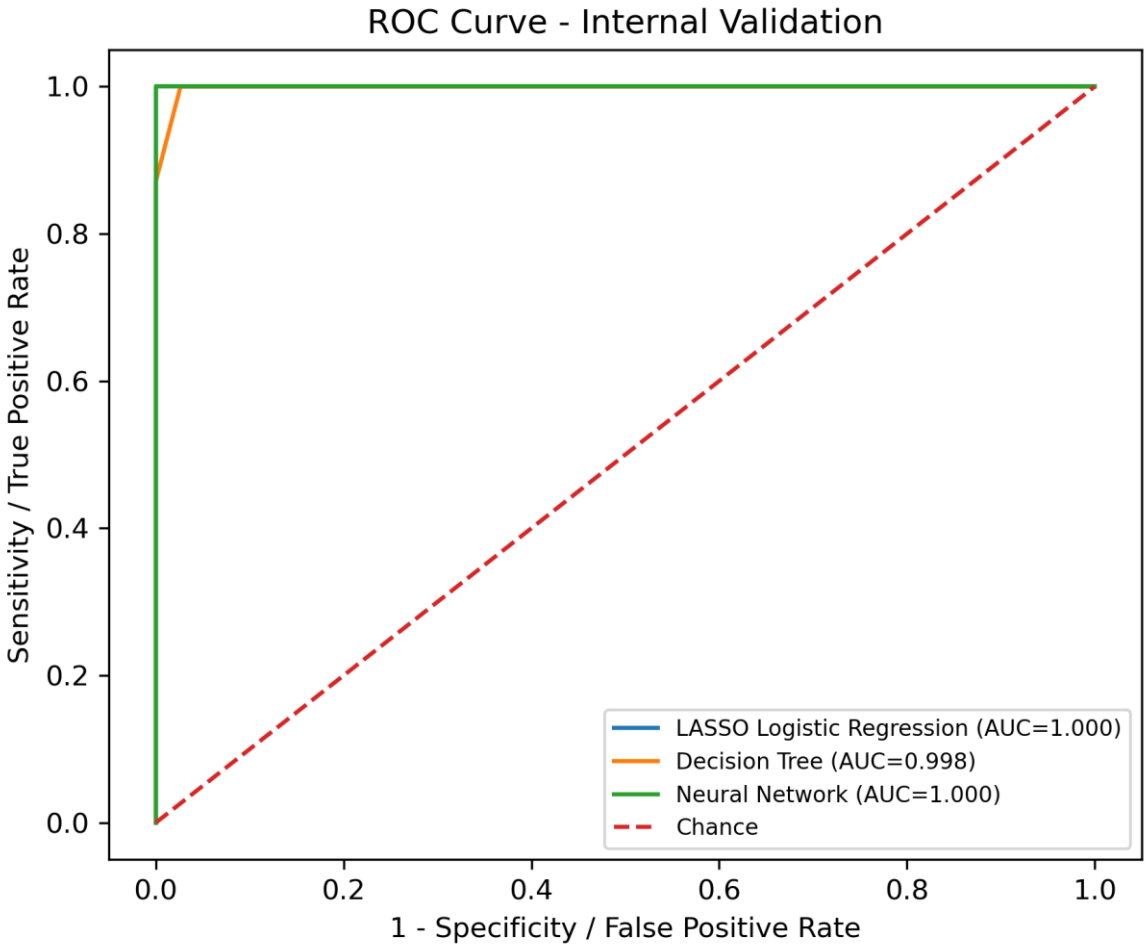
Tabel 7. Calibration intercept dan calibration slope

Model	Calibration intercept	Calibration slope
LASSO Logistic Regression	0.910	4.002
Decision Tree	-0.052	2.617
Neural Network	6.546	19.519

Hasil internal validation menunjukkan bahwa LASSO Logistic Regression memperoleh accuracy 1,000, sensitivity 1,000, specificity 1,000, AUC 1,000, dan Brier score 0,0016. Decision Tree juga menunjukkan hasil sangat baik dengan accuracy 0,990 dan AUC 0,998, tetapi terdapat satu false positive. Neural Network memperoleh AUC 1,000, tetapi memiliki dua false negative dan Brier score lebih besar, yaitu 0,0665.

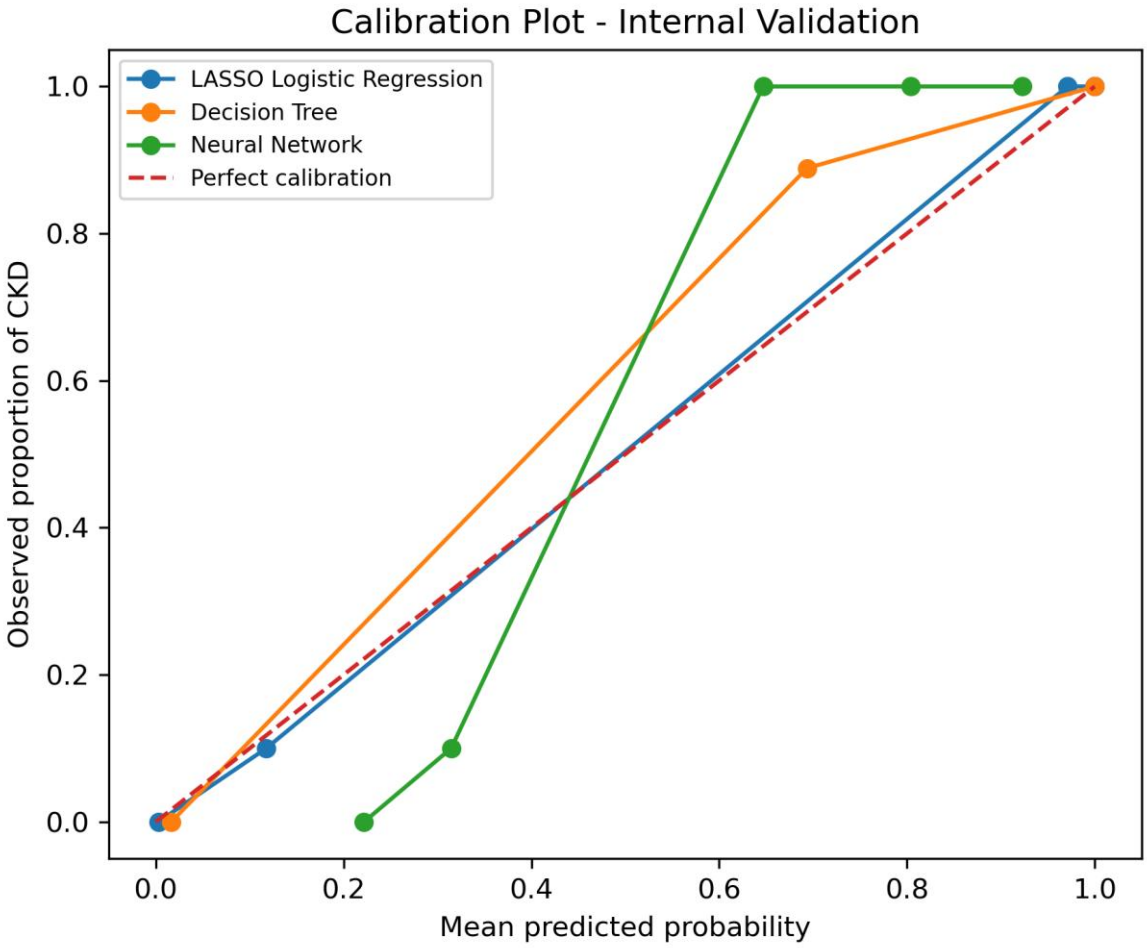
Berdasarkan kombinasi tersebut, LASSO Logistic Regression menjadi model dengan keseimbangan terbaik antara diskriminasi, ketepatan klasifikasi, kalibrasi probabilitas, dan interpretabilitas. Dengan demikian, nilai *calibration intercept* dan *calibration slope* tidak sebaiknya ditafsirkan secara terpisah sebagai bukti mutlak kualitas kalibrasi model. Interpretasi perlu dilakukan secara hati-hati karena *internal validation set* memiliki ukuran sampel yang relatif terbatas, sementara beberapa model menghasilkan probabilitas prediksi yang sangat ekstrem. Kondisi tersebut dapat menyebabkan estimasi parameter kalibrasi menjadi kurang stabil.

4.3 Visualisasi Evaluasi Model



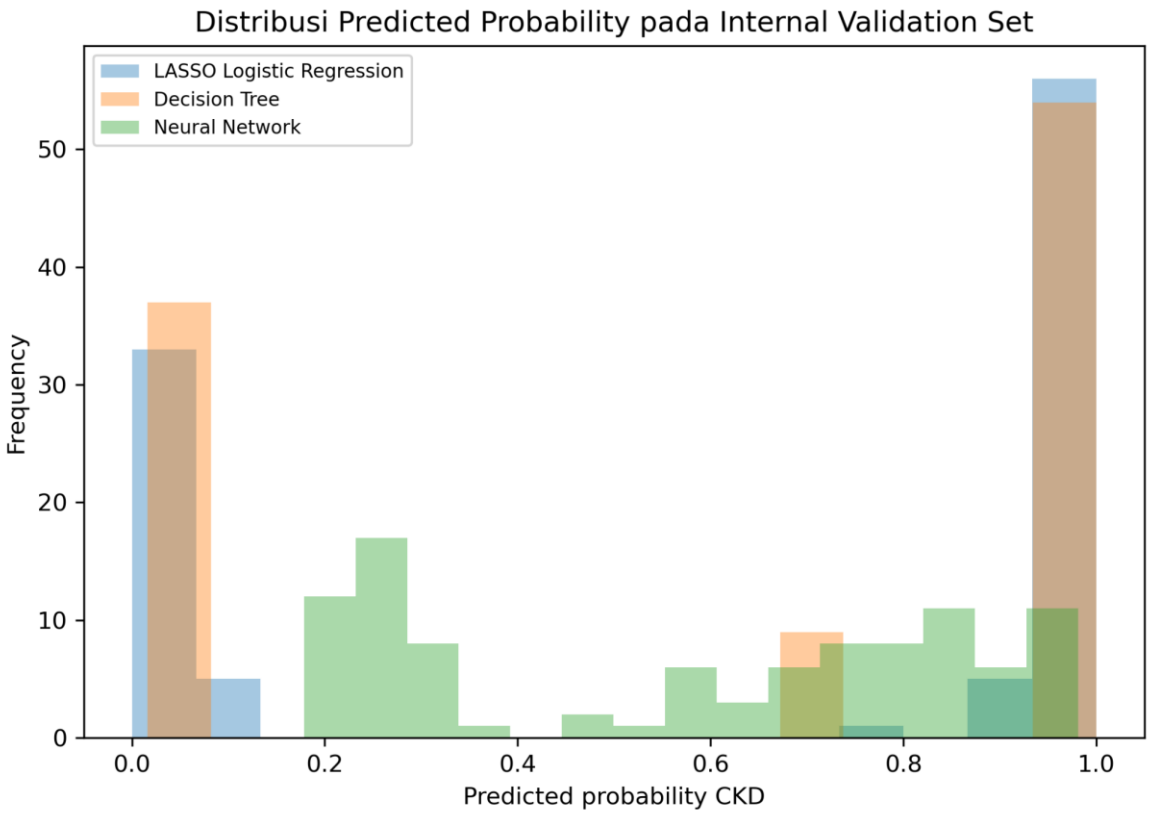
Gambar 1. ROC curve pada internal validation set

ROC curve memperlihatkan kemampuan diskriminasi yang sangat tinggi pada semua model. LASSO Logistic Regression dan Neural Network mencapai AUC 1,000, sedangkan Decision Tree mencapai AUC 0,998.



Gambar 2. Calibration plot pada internal validation set

Calibration plot memperlihatkan hubungan antara probabilitas prediksi rata-rata dan proporsi CKD yang diamati. Secara umum, LASSO lebih konsisten dengan Brier score paling rendah, sedangkan Neural Network menunjukkan variasi probabilitas yang lebih lebar.



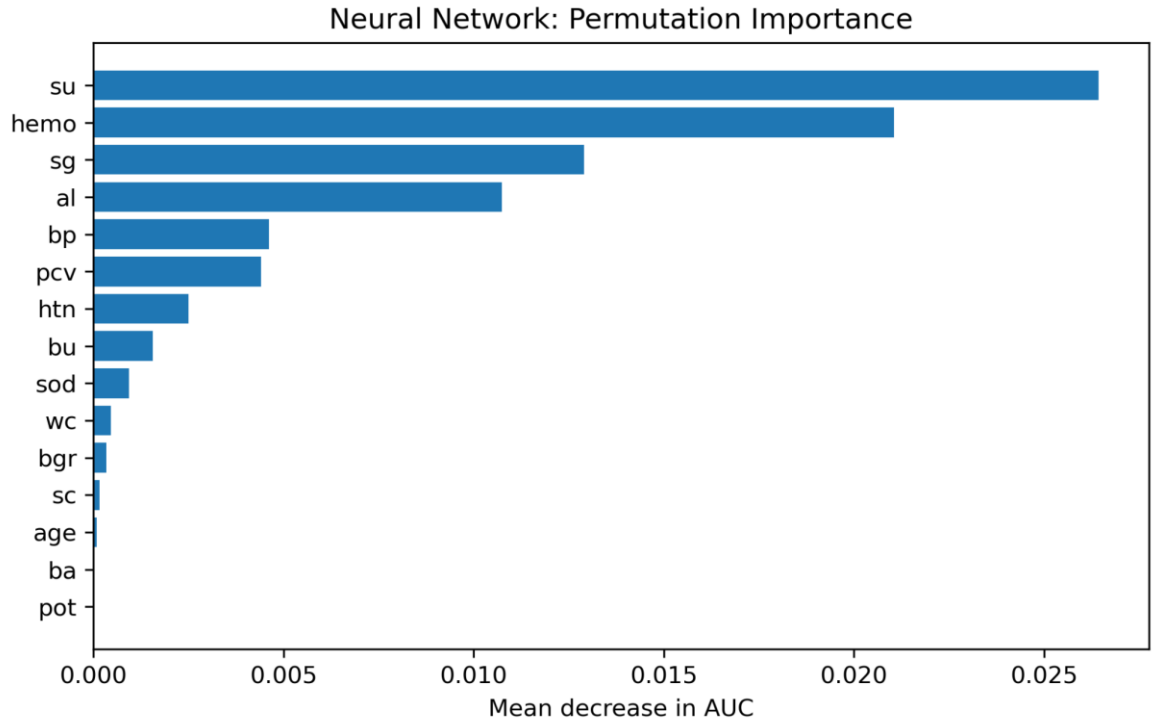
Gambar 3. Distribusi predicted probability pada internal validation set

Distribusi predicted probability menunjukkan bahwa LASSO dan Decision Tree cenderung menghasilkan probabilitas yang terpisah jelas antara kelompok rendah dan tinggi, sementara Neural Network menghasilkan sebaran probabilitas yang lebih bervariasi.

Gambar 4. Confusion matrix LASSO Logistic Regression

Tabel 10. Permutation importance pada Neural Network

Feature	Permutation importance mean	Permutation importance SD
su	0.0264	0.0044
hemo	0.0211	0.0074
sg	0.0129	0.0041
al	0.0107	0.0036
bp	0.0046	0.0036
pcv	0.0044	0.0037
htn	0.0025	0.0013
bu	0.0016	0.0022
sod	0.0009	0.0007
wc	0.0005	0.0010
bgr	0.0003	0.0009
sc	0.0002	0.0003



Gambar 10. Permutation importance Neural Network

Permutation importance menunjukkan bahwa variabel su, hemo, sg, dan al merupakan prediktor dengan pengaruh relatif terbesar pada Neural Network. Meskipun demikian, interpretasi Neural Network tetap lebih terbatas dibandingkan LASSO karena kontribusi prediktor tidak langsung berbentuk koefisien yang mudah dijelaskan.

VI. PEMILIHAN MODEL TERBAIK, KESIMPULAN, DAN KETERBATASAN

6.1 Pemilihan Model Terbaik

Model terbaik pada pengerjaan ini adalah LASSO Logistic Regression. Dasar pemilihannya adalah kombinasi antara performa diskriminasi, kalibrasi, dan interpretabilitas. Pada internal validation set, LASSO menghasilkan accuracy 1,000, sensitivity 1,000, specificity 1,000, AUC 1,000, dan Brier score paling rendah, yaitu 0,0016.

Decision Tree menjadi model alternatif yang sangat baik karena mudah dijelaskan melalui aturan klinis sederhana, tetapi performanya sedikit lebih rendah karena terdapat satu false positive. Neural

Network memiliki diskriminasi tinggi, tetapi kurang dipilih sebagai model utama karena interpretabilitasnya lebih rendah dan Brier score lebih besar daripada LASSO.

6.2 Kesimpulan

Pemodelan prediksi CKD telah dilakukan menggunakan LASSO Logistic Regression, Decision Tree, dan Neural Network dengan pendekatan development dan internal validation. Seluruh model menunjukkan performa sangat baik karena dataset CKD memiliki pola klinis dan laboratorium yang cukup kuat untuk membedakan CKD dan Not CKD.

LASSO Logistic Regression dipilih sebagai model terbaik karena memberikan performa klasifikasi paling baik sekaligus tetap interpretable. Model ini mempertahankan prediktor yang secara klinis relevan, antara lain specific gravity, hemoglobin, albumin, diabetes mellitus, appetite, hipertensi, packed cell volume, serum creatinine, red blood cell count, edema, sugar, blood glucose random, sodium, blood pressure, dan blood urea.

6.3 Keterbatasan dan Rekomendasi

Hasil ini merupakan internal validation sehingga belum cukup untuk klaim generalisasi luas. Nilai performa yang sangat tinggi dapat dipengaruhi oleh karakteristik dataset, ukuran sampel, pola missing value, dan kemungkinan pemisahan kelas yang kuat pada data internal.

Sebelum digunakan untuk pengambilan keputusan klinis nyata, model masih perlu diuji menggunakan external validation pada data dari sumber, waktu, atau populasi yang berbeda. External validation penting untuk menilai apakah performa model tetap stabil ketika diterapkan pada konteks klinis yang tidak digunakan selama pengembangan model.

LAMPIRAN

```
# Cell 1
# 0.1 Import library utama

import os
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import display, Image as IPyImage

from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, brier_score_loss, balanced_accuracy_score
)
from sklearn.calibration import calibration_curve
from sklearn.inspection import permutation_importance

print('Library berhasil dipanggil.')


# Cell 2
# 0.2 Menetapkan random state

RANDOM_STATE = 123
```

```
print('Random state:', RANDOM_STATE)


# Cell 3
# 0.3 Menentukan folder kerja dan path dataset

WINDOWS_PROJECT_DIR = r"D:\03_Univ\02_Magister\2nd Semester\03_Data Mining\UAS\Final Examination (Class Work
Only)\Mohammad Maliki Rafli_291251025_UAS_DMML"

if os.path.isdir(WINDOWS_PROJECT_DIR):
    PROJECT_DIR = WINDOWS_PROJECT_DIR
else:
    PROJECT_DIR = os.getcwd()

DATA_CANDIDATES = [
    os.path.join(PROJECT_DIR, 'data_uas_kidney_disease.csv'),
    os.path.join(os.path.dirname(PROJECT_DIR), 'data_uas_kidney_disease.csv'),
    r"D:\03_Univ\02_Magister\2nd Semester\03_Data Mining\UAS\Final Examination (Class Work
Only)\data_uas_kidney_disease.csv",
    'data_uas_kidney_disease.csv',
    '../data_uas_kidney_disease.csv',
    '/mnt/data/data_uas_kidney_disease.csv'
]

DATA_PATH = None
for candidate in DATA_CANDIDATES:
    if os.path.exists(candidate):
        DATA_PATH = candidate
        break

if DATA_PATH is None:
    raise FileNotFoundError(
        'File data_uas_kidney_disease.csv tidak ditemukan. '
        'Pastikan file dataset berada di folder kerja atau satu folder di atas notebook.'
    )

print('Folder kerja:')
print(PROJECT_DIR)

print('\nDataset yang digunakan:')
print(DATA_PATH)


# Cell 4
# 0.4 Membuat folder output

BASE_DIR = os.path.join(PROJECT_DIR, 'outputs')
TABLE_DIR = os.path.join(BASE_DIR, 'tables')
FIGURE_DIR = os.path.join(BASE_DIR, 'figures')

os.makedirs(TABLE_DIR, exist_ok=True)
os.makedirs(FIGURE_DIR, exist_ok=True)

print('Folder output utama:', BASE_DIR)
print('Folder output tabel:', TABLE_DIR)
print('Folder output gambar:', FIGURE_DIR)


# Cell 5
# 0.5 Mengatur tampilan tabel

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 120)
pd.set_option('display.float_format', lambda x: f'{x:.4f}')

print('Pengaturan tampilan selesai.')


# Cell 6
# 1.1 Membaca dataset CKD

raw = pd.read_csv(DATA_PATH)
df = raw.copy()

print('Ukuran data awal:', df.shape)
```

```
# Cell 7
# 1.2 Menampilkan nama kolom dataset

print('Daftar kolom:')
print(df.columns.tolist())


# Cell 8
# 1.3 Menampilkan baris atas

display(df.head())


# Cell 9
# 1.4 Menampilkan tipe data awal

tipe_data_awal = df.dtypes.to_frame('Tipe data awal')
display(tipe_data_awal)


# Cell 10
# 2.1 Merapikan nama kolom

df.columns = df.columns.str.strip().str.lower()

print('Nama kolom setelah dirapikan:')
print(df.columns.tolist())


# Cell 11
# 2.2 Membersihkan nilai kategorikal dan tanda missing value

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = df[col].astype(str).str.strip()
        df[col] = df[col].replace({'?': np.nan, 'nan': np.nan, '': np.nan})

print('Cleaning nilai kategorikal selesai.')


# Cell 12
# 2.3 Membersihkan dan mengkodekan target outcome
# CKD = 1, Not CKD = 0

df['classification'] = df['classification'].str.strip().str.lower()
df['target_ckd'] = df['classification'].map({'ckd': 1, 'notckd': 0})

display(df['classification'].value_counts(dropna=False).to_frame('n'))


# Cell 13
# 2.4 Menampilkan distribusi target dalam kode numerik

target_distribution =
df['target_ckd'].value_counts(dropna=False).rename_axis('Target_CKD').reset_index(name='n')
target_distribution['%'] = (target_distribution['n'] / target_distribution['n'].sum() * 100).round(1)

display(target_distribution)


# Cell 14
# 2.5 Mengecek missing value pada target

print('Jumlah missing pada target_ckd:', df['target_ckd'].isna().sum())


# Cell 15
# 2.6 Memisahkan prediktor dan target

X = df.drop(columns=['id', 'classification', 'target_ckd'])
y = df['target_ckd'].astype(int)

print('Ukuran prediktor X:', X.shape)
print('Ukuran target y:', y.shape)
```

```
# Cell 16
# 2.7 Menetapkan variabel kategorikal

categorical_reference = ['rbc', 'pc', 'pcc', 'ba', 'htn', 'dm', 'cad', 'appet', 'pe', 'ane']

print('Variabel kategorikal:')
print(categorical_reference)


# Cell 17
# 2.8 Mengubah variabel selain kategorikal menjadi numerik

for col in X.columns:
    if col not in categorical_reference:
        X[col] = pd.to_numeric(X[col], errors='coerce')

print('Konversi numerik selesai.')


# Cell 18
# 2.9 Menentukan variabel numerik dan kategorikal

numeric_features = [col for col in X.columns if col not in categorical_reference]
categorical_features = categorical_reference.copy()

print('Jumlah prediktor numerik:', len(numeric_features))
print(numeric_features)
print()
print('Jumlah prediktor kategorikal:', len(categorical_features))
print(categorical_features)


# Cell 19
# 2.10 Menyusun tabel missing value

missing_table = pd.DataFrame({
    'Variabel': X.columns,
    'Missing_n': X.isna().sum().values,
    'Missing_%': np.round(X.isna().mean().values * 100, 1)
}).sort_values('Missing_%', ascending=False)

missing_table.to_csv(os.path.join(TABLE_DIR, 'missingness_table.csv'), index=False)

display(missing_table)


# Cell 20
# 3.1 Membagi dataset secara stratified

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y,
    test_size=0.25,
    random_state=RANDOM_STATE,
    stratify=y
)

print('Development set:', X_train.shape)
print('Internal validation set:', X_valid.shape)


# Cell 21
# 3.2 Menampilkan distribusi target pada setiap set

split_table = pd.DataFrame({
    'Set': ['Development', 'Internal validation'],
    'n': [len(y_train), len(y_valid)],
    'CKD_n': [int(y_train.sum()), int(y_valid.sum())],
    'Not_CKD_n': [int((y_train == 0).sum()), int((y_valid == 0).sum())],
    'CKD_%': [round(y_train.mean() * 100, 1), round(y_valid.mean() * 100, 1)]
})

split_table.to_csv(os.path.join(TABLE_DIR, 'data_split_table.csv'), index=False)

display(split_table)


# Cell 22
```

```
# 4.1 Membuat fungsi OneHotEncoder agar kompatibel pada beberapa versi scikit-learn

def make_onehot_encoder():
    try:
        return OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
    except TypeError:
        return OneHotEncoder(drop='first', handle_unknown='ignore', sparse=False)

print('Fungsi OneHotEncoder siap.')


# Cell 23
# 4.2 Membuat preprocessing untuk LASSO Logistic Regression dan Neural Network

preprocess_scaled = ColumnTransformer(
    transformers=[
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), numeric_features),
        ('cat', Pipeline([
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', make_onehot_encoder())
        ]), categorical_features)
    ]
)

print('Preprocessing scaled siap.')


# Cell 24
# 4.3 Membuat preprocessing untuk Decision Tree

preprocess_tree = ColumnTransformer(
    transformers=[
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median'))
        ]), numeric_features),
        ('cat', Pipeline([
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', make_onehot_encoder())
        ]), categorical_features)
    ]
)

print('Preprocessing Decision Tree siap.')


# Cell 25
# 4.4 Menetapkan skema cross-validation

cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)

print('Cross-validation menggunakan StratifiedKFold 3-fold.')


# Cell 26
# 5.1 Membuat pipeline model LASSO Logistic Regression

lasso_pipeline = Pipeline([
    ('preprocess', preprocess_scaled),
    ('model', LogisticRegression(
        penalty='l1',
        solver='saga',
        max_iter=20000,
        random_state=RANDOM_STATE,
        class_weight='balanced'
    ))
])

print('Pipeline LASSO Logistic Regression siap.')


# Cell 27
# 5.2 Membuat pipeline model Decision Tree

tree_pipeline = Pipeline([
```

```
    ('preprocess', preprocess_tree),
    ('model', DecisionTreeClassifier(
        random_state=RANDOM_STATE,
        class_weight='balanced'
    ))
])

print('Pipeline Decision Tree siap.')


# Cell 28
# 5.3 Membuat pipeline model Neural Network

nn_pipeline = Pipeline([
    ('preprocess', preprocess_scaled),
    ('model', MLPClassifier(
        random_state=RANDOM_STATE,
        max_iter=1200,
        early_stopping=True,
        validation_fraction=0.15,
        n_iter_no_change=20
    ))
])

print('Pipeline Neural Network siap.')


# Cell 29
# 5.4 Menetapkan grid hyperparameter

grids = {
    'LASSO Logistic Regression': {
        'model__C': [0.01, 0.03, 0.1, 0.3, 1.0]
    },
    'Decision Tree': {
        'model__criterion': ['gini', 'entropy'],
        'model__max_depth': [2, 3, 4],
        'model__min_samples_leaf': [1, 5, 10],
        'model__ccp_alpha': [0.0, 0.005]
    },
    'Neural Network': {
        'model__hidden_layer_sizes': [(8,), (16,), (16, 8)],
        'model__alpha': [0.001, 0.01],
        'model__activation': ['relu']
    }
}

print('Grid hyperparameter siap.')


# Cell 30
# 5.5 Hyperparameter tuning LASSO Logistic Regression

search_lasso = GridSearchCV(
    estimator=lasso_pipeline,
    param_grid=grids['LASSO Logistic Regression'],
    scoring='roc_auc',
    cv=cv,
    n_jobs=1,
    refit=True,
    return_train_score=True
)

search_lasso.fit(X_train, y_train)

print('Best parameter LASSO:')
print(search_lasso.best_params_)
print('Mean CV AUC:', round(search_lasso.best_score_, 4))


# Cell 31
# 5.6 Hyperparameter tuning Decision Tree

search_tree = GridSearchCV(
    estimator=tree_pipeline,
    param_grid=grids['Decision Tree'],
    scoring='roc_auc',
```

```
    cv=cv,
    n_jobs=1,
    refit=True,
    return_train_score=True
)

search_tree.fit(X_train, y_train)

print('Best parameter Decision Tree:')
print(search_tree.best_params_)
print('Mean CV AUC:', round(search_tree.best_score_, 4))


# Cell 32
# 5.7 Hyperparameter tuning Neural Network

search_nn = GridSearchCV(
    estimator=nn_pipeline,
    param_grid=grids['Neural Network'],
    scoring='roc_auc',
    cv=cv,
    n_jobs=1,
    refit=True,
    return_train_score=True
)

search_nn.fit(X_train, y_train)

print('Best parameter Neural Network:')
print(search_nn.best_params_)
print('Mean CV AUC:', round(search_nn.best_score_, 4))


# Cell 33
# 5.8 Menyusun ringkasan hasil hyperparameter tuning

search_objects = {
    'LASSO Logistic Regression': search_lasso,
    'Decision Tree': search_tree,
    'Neural Network': search_nn
}

cv_summary = []
for name, search in search_objects.items():
    cv_summary.append({
        'Model': name,
        'Best_params': str(search.best_params_),
        'Mean_CV_AUC_at_best': search.best_score_,
        'Mean_train_AUC_at_best': search.cv_results_['mean_train_score'][search.best_index_]
    })

cv_results = pd.DataFrame(cv_summary)
cv_results.to_csv(os.path.join(TABLE_DIR, 'cv_results_summary.csv'), index=False)

display(cv_results)


# Cell 34
# 5.9 Menggabungkan semua model terbaik

best_estimators = {
    'LASSO Logistic Regression': search_lasso.best_estimator_,
    'Decision Tree': search_tree.best_estimator_,
    'Neural Network': search_nn.best_estimator_
}

print('Model terbaik dari development set:')
print(list(best_estimators.keys()))


# Cell 35
# 6.1 Membuat fungsi evaluasi model

def evaluate_model(name, model, X_valid, y_valid, threshold=0.5):
    prob = model.predict_proba(X_valid)[:, 1]
    pred = (prob >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_valid, pred).ravel()
```

```
    return {
        'Model': name,
        'Threshold': threshold,
        'TN': tn,
        'FP': fp,
        'FN': fn,
        'TP': tp,
        'Accuracy': accuracy_score(y_valid, pred),
        'Balanced_accuracy': balanced_accuracy_score(y_valid, pred),
        'Sensitivity_Recall_CKD': recall_score(y_valid, pred),
        'Specificity_Not_CKD': tn / (tn + fp),
        'Precision_PPV': precision_score(y_valid, pred, zero_division=0),
        'F1_score': f1_score(y_valid, pred, zero_division=0),
        'AUC': roc_auc_score(y_valid, prob),
        'Brier_score': brier_score_loss(y_valid, prob)
    }

print('Fungsi evaluasi siap.')


# Cell 36
# 6.2 Evaluasi LASSO Logistic Regression

lasso_eval = evaluate_model(
    'LASSO Logistic Regression',
    best_estimators['LASSO Logistic Regression'],
    X_valid,
    y_valid
)

display(pd.DataFrame([lasso_eval]))


# Cell 37
# 6.3 Evaluasi Decision Tree

tree_eval = evaluate_model(
    'Decision Tree',
    best_estimators['Decision Tree'],
    X_valid,
    y_valid
)

display(pd.DataFrame([tree_eval]))


# Cell 38
# 6.4 Evaluasi Neural Network

nn_eval = evaluate_model(
    'Neural Network',
    best_estimators['Neural Network'],
    X_valid,
    y_valid
)

display(pd.DataFrame([nn_eval]))


# Cell 39
# 6.5 Menggabungkan hasil evaluasi semua model

eval_df = pd.DataFrame([lasso_eval, tree_eval, nn_eval]).sort_values(
    ['AUC', 'Brier_score'],
    ascending=[False, True]
)

eval_df.to_csv(os.path.join(TABLE_DIR, 'internal_validation_metrics.csv'), index=False)

display(eval_df)


# Cell 40
# 6.6 Menyimpan probabilitas prediksi dan kelas prediksi

prediction_rows = []
```

```
for name, model in best_estimators.items():
    prob = model.predict_proba(X_valid)[:, 1]
    pred = (prob >= 0.5).astype(int)
    prediction_rows.append(pd.DataFrame({
        'Model': name,
        'y_true': y_valid.values,
        'prob_ckd': prob,
        'pred_ckd': pred
    }))

prediction_df = pd.concat(prediction_rows, ignore_index=True)
prediction_df.to_csv(os.path.join(TABLE_DIR, 'validation_predictions.csv'), index=False)

display(prediction_df.head(12))


# Cell 41
# 7.1 Membuat fungsi bootstrap confidence interval

def bootstrap_metric_ci(y_true, prob, metric_func, n_boot=500, random_state=123):
    rng = np.random.default_rng(random_state)
    y_true = np.asarray(y_true)
    prob = np.asarray(prob)
    values = []

    for _ in range(n_boot):
        idx = rng.choice(len(y_true), size=len(y_true), replace=True)
        if len(np.unique(y_true[idx])) < 2:
            continue
        values.append(metric_func(y_true[idx], prob[idx]))

    return np.percentile(values, [2.5, 97.5])

print('Fungsi bootstrap CI siap.')


# Cell 42
# 7.2 Menghitung bootstrap CI untuk AUC dan Accuracy

boot_rows = []

for name, model in best_estimators.items():
    prob = model.predict_proba(X_valid)[:, 1]
    pred = (prob >= 0.5).astype(int)

    auc_low, auc_high = bootstrap_metric_ci(
        y_valid.values,
        prob,
        roc_auc_score,
        n_boot=500,
        random_state=RANDOM_STATE
    )

    acc_low, acc_high = bootstrap_metric_ci(
        y_valid.values,
        prob,
        lambda yt, pr: accuracy_score(yt, (pr >= 0.5).astype(int)),
        n_boot=500,
        random_state=RANDOM_STATE
    )

    boot_rows.append({
        'Model': name,
        'AUC_95CI_low': auc_low,
        'AUC_95CI_high': auc_high,
        'Accuracy_95CI_low': acc_low,
        'Accuracy_95CI_high': acc_high
    })

boot_df = pd.DataFrame(boot_rows)
boot_df.to_csv(os.path.join(TABLE_DIR, 'bootstrap_ci_metrics.csv'), index=False)

display(boot_df)


# Cell 43
# 7.3 Membuat fungsi calibration intercept dan calibration slope
```

```
def calibration_intercept_slope(y_true, prob):
    eps = 1e-6
    prob = np.clip(prob, eps, 1 - eps)
    logit_prob = np.log(prob / (1 - prob)).reshape(-1, 1)

    cal_model = LogisticRegression(penalty=None, solver='lbfgs', max_iter=1000)
    try:
        cal_model.fit(logit_prob, y_true)
    except TypeError:
        cal_model = LogisticRegression(penalty='none', solver='lbfgs', max_iter=1000)
        cal_model.fit(logit_prob, y_true)

    intercept = cal_model.intercept_[0]
    slope = cal_model.coef_[0][0]

    return intercept, slope

print('Fungsi calibration intercept dan slope siap.')


# Cell 44
# 7.4 Menghitung calibration intercept dan calibration slope

cal_rows = []

for name, model in best_estimators.items():
    prob = model.predict_proba(X_valid)[:, 1]
    intercept, slope = calibration_intercept_slope(y_valid.values, prob)
    cal_rows.append({
        'Model': name,
        'Calibration_intercept': intercept,
        'Calibration_slope': slope
    })

cal_df = pd.DataFrame(cal_rows)
cal_df.to_csv(os.path.join(TABLE_DIR, 'calibration_intercept_slope.csv'), index=False)

display(cal_df)


# Cell 45
# 8.1 Membuat ROC curve internal validation

plt.figure(figsize=(6, 5))

for name, model in best_estimators.items():
    prob = model.predict_proba(X_valid)[:, 1]
    fpr, tpr, _ = roc_curve(y_valid, prob)
    auc_value = roc_auc_score(y_valid, prob)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc_value:.3f})')

plt.plot([0, 1], [0, 1], linestyle='--', label='Chance')
plt.xlabel('1 - Specificity / False Positive Rate')
plt.ylabel('Sensitivity / True Positive Rate')
plt.title('ROC Curve - Internal Validation')
plt.legend(loc='lower right', fontsize=8)
plt.tight_layout()

roc_path = os.path.join(FIGURE_DIR, 'roc_curve_internal_validation.png')
plt.savefig(roc_path, dpi=300, bbox_inches='tight')
plt.show()

print('ROC curve disimpan di:', roc_path)


# Cell 46
# 8.2 Membuat calibration plot internal validation

plt.figure(figsize=(6, 5))

for name, model in best_estimators.items():
    prob = model.predict_proba(X_valid)[:, 1]
    frac_pos, mean_pred = calibration_curve(y_valid, prob, n_bins=5, strategy='quantile')
    plt.plot(mean_pred, frac_pos, marker='o', label=name)

plt.plot([0, 1], [0, 1], linestyle='--', label='Perfect calibration')
```

```
plt.xlabel('Mean predicted probability')
plt.ylabel('Observed proportion of CKD')
plt.title('Calibration Plot - Internal Validation')
plt.legend(fontsize=8)
plt.tight_layout()

calibration_path = os.path.join(FIGURE_DIR, 'calibration_plot_internal_validation.png')
plt.savefig(calibration_path, dpi=300, bbox_inches='tight')
plt.show()

print('Calibration plot disimpan di:', calibration_path)


# Cell 47
# 8.3 Membuat fungsi confusion matrix

def plot_confusion_matrix_model(name, model, X_valid, y_valid):
    prob = model.predict_proba(X_valid)[:, 1]
    pred = (prob >= 0.5).astype(int)
    cm = confusion_matrix(y_valid, pred, labels=[0, 1])

    fig, ax = plt.subplots(figsize=(5, 4))
    ax.imshow(cm)
    ax.set_title(f'Confusion Matrix - {name}')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
    ax.set_xticks([0, 1])
    ax.set_xticklabels(['Not CKD', 'CKD'])
    ax.set_yticks([0, 1])
    ax.set_yticklabels(['Not CKD', 'CKD'])

    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, cm[i, j], ha='center', va='center')

    plt.tight_layout()

    safe_name = name.lower().replace(' ', '_').replace('-', '_')
    fig_path = os.path.join(FIGURE_DIR, f'confusion_matrix_{safe_name}.png')
    plt.savefig(fig_path, dpi=300, bbox_inches='tight')
    plt.show()

    print('Confusion matrix disimpan di:', fig_path)

print('Fungsi confusion matrix siap.')


# Cell 48
# 8.4 Confusion matrix LASSO Logistic Regression

plot_confusion_matrix_model(
    'LASSO Logistic Regression',
    best_estimators['LASSO Logistic Regression'],
    X_valid,
    y_valid
)


# Cell 49
# 8.5 Confusion matrix Decision Tree

plot_confusion_matrix_model(
    'Decision Tree',
    best_estimators['Decision Tree'],
    X_valid,
    y_valid
)


# Cell 50
# 8.6 Confusion matrix Neural Network

plot_confusion_matrix_model(
    'Neural Network',
    best_estimators['Neural Network'],
    X_valid,
    y_valid
```

```
)


# Cell 51
# 8.7 Visualisasi distribusi predicted probability

plt.figure(figsize=(7, 5))

for name, model in best_estimators.items():
    prob = model.predict_proba(X_valid)[:, 1]
    plt.hist(prob, bins=15, alpha=0.4, label=name)

plt.xlabel('Predicted probability CKD')
plt.ylabel('Frequency')
plt.title('Distribusi Predicted Probability pada Internal Validation Set')
plt.legend(fontsize=8)
plt.tight_layout()

prob_dist_path = os.path.join(FIGURE_DIR, 'predicted_probability_distribution.png')
plt.savefig(prob_dist_path, dpi=300, bbox_inches='tight')
plt.show()

print('Distribusi predicted probability disimpan di:', prob_dist_path)


# Cell 52
# 9.1 Membuat fungsi untuk mengambil nama fitur setelah preprocessing

def get_feature_names(preprocess):
    names = []
    names.extend(numeric_features)
    onehot = preprocess.named_transformers_['cat'].named_steps['onehot']
    names.extend(onehot.get_feature_names_out(categorical_features).tolist())
    return names

print('Fungsi nama fitur siap.')


# Cell 53
# 9.2 Menyusun tabel koefisien LASSO Logistic Regression

lasso = best_estimators['LASSO Logistic Regression']
lasso_feature_names = get_feature_names(lasso.named_steps['preprocess'])
lasso_coef = lasso.named_steps['model'].coef_[0]

coef_df = pd.DataFrame({
    'Feature': lasso_feature_names,
    'Coefficient': lasso_coef
})

coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()
coef_df = coef_df.sort_values('Abs_Coefficient', ascending=False)
coef_df.to_csv(os.path.join(TABLE_DIR, 'lasso_coefficients.csv'), index=False)

display(coef_df.head(20))


# Cell 54
# 9.3 Menampilkan prediktor terpilih LASSO

selected_lasso = coef_df[coef_df['Abs_Coefficient'] > 1e-8].copy()
selected_lasso.to_csv(os.path.join(TABLE_DIR, 'lasso_selected_predictors.csv'), index=False)

display(selected_lasso)


# Cell 55
# 9.4 Membuat tabel score seperti nomogram sederhana

nomogram_table = selected_lasso.copy()
nomogram_table['Nomogram_points_like'] = (
    nomogram_table['Abs_Coefficient'] / nomogram_table['Abs_Coefficient'].max() * 100
).round(1)
nomogram_table['Direction'] = np.where(
    nomogram_table['Coefficient'] > 0,
    'Meningkatkan peluang CKD',
    'Menurunkan peluang CKD'
```

```
)

nomogram_table = nomogram_table[
    ['Feature', 'Coefficient', 'Abs_Coefficient', 'Nomogram_points_like', 'Direction']
]

nomogram_table.to_csv(os.path.join(TABLE_DIR, 'lasso_nomogram_like_score_table.csv'), index=False)

display(nomogram_table)


# Cell 56
# 9.5 Membuat visualisasi koefisien LASSO

plot_coef = selected_lasso.head(20).sort_values('Coefficient')

plt.figure(figsize=(7, max(4, 0.3 * len(plot_coef))))
plt.barh(plot_coef['Feature'], plot_coef['Coefficient'])
plt.axvline(0, linestyle='--')
plt.title('LASSO Logistic Regression: Selected Predictors')
plt.xlabel('Coefficient (log-odds CKD)')
plt.tight_layout()

lasso_plot_path = os.path.join(FIGURE_DIR, 'lasso_selected_coefficients.png')
plt.savefig(lasso_plot_path, dpi=300, bbox_inches='tight')
plt.show()

print('Plot koefisien LASSO disimpan di:', lasso_plot_path)


# Cell 57
# 10.1 Mengambil nama fitur setelah preprocessing Decision Tree

best_tree = best_estimators['Decision Tree']
tree_feature_names = get_feature_names(best_tree.named_steps['preprocess'])

print('Jumlah fitur setelah preprocessing:', len(tree_feature_names))


# Cell 58
# 10.2 Membuat diagram Decision Tree

plt.figure(figsize=(18, 9))

plot_tree(
    best_tree.named_steps['model'],
    feature_names=tree_feature_names,
    class_names=['Not CKD', 'CKD'],
    filled=True,
    impurity=True,
    rounded=True,
    max_depth=4,
    fontsize=8
)

plt.title('Decision Tree Diagram')
plt.tight_layout()

tree_diagram_path = os.path.join(FIGURE_DIR, 'decision_tree_diagram.png')
plt.savefig(tree_diagram_path, dpi=300, bbox_inches='tight')
plt.show()

print('Diagram Decision Tree disimpan di:', tree_diagram_path)


# Cell 59
# 10.3 Menampilkan aturan ringkas Decision Tree

tree_rules = export_text(
    best_tree.named_steps['model'],
    feature_names=tree_feature_names,
    max_depth=5
)

print(tree_rules)
```

```
# Cell 60
# 10.4 Menyusun feature importance Decision Tree

tree_importance = pd.DataFrame({
    'Feature': tree_feature_names,
    'Importance': best_tree.named_steps['model'].feature_importances_
}).sort_values('Importance', ascending=False)

tree_importance.to_csv(os.path.join(TABLE_DIR, 'decision_tree_feature_importance.csv'), index=False)

display(tree_importance.head(15))


# Cell 61
# 10.5 Membuat visualisasi feature importance Decision Tree

plot_tree_importance = tree_importance.head(15).sort_values('Importance')

plt.figure(figsize=(7, max(4, 0.3 * len(plot_tree_importance))))
plt.barh(plot_tree_importance['Feature'], plot_tree_importance['Importance'])
plt.title('Decision Tree: Feature Importance')
plt.xlabel('Importance')
plt.tight_layout()

tree_importance_path = os.path.join(FIGURE_DIR, 'decision_tree_feature_importance.png')
plt.savefig(tree_importance_path, dpi=300, bbox_inches='tight')
plt.show()

print('Plot feature importance Decision Tree disimpan di:', tree_importance_path)


# Cell 62
# 11.1 Menghitung permutation importance Neural Network

nn = best_estimators['Neural Network']

perm = permutation_importance(
    nn,
    X_valid,
    y_valid,
    scoring='roc_auc',
    n_repeats=10,
    random_state=RANDOM_STATE,
    n_jobs=1
)

nn_importance = pd.DataFrame({
    'Feature': X.columns,
    'Permutation_importance_mean': perm.importances_mean,
    'Permutation_importance_sd': perm.importances_std
}).sort_values('Permutation_importance_mean', ascending=False)

nn_importance.to_csv(os.path.join(TABLE_DIR, 'neural_network_permutation_importance.csv'), index=False)

display(nn_importance.head(15))


# Cell 63
# 11.2 Membuat visualisasi permutation importance Neural Network

plot_perm = nn_importance.head(15).sort_values('Permutation_importance_mean')

plt.figure(figsize=(7, max(4, 0.3 * len(plot_perm))))
plt.barh(plot_perm['Feature'], plot_perm['Permutation_importance_mean'])
plt.title('Neural Network: Permutation Importance')
plt.xlabel('Mean decrease in AUC')
plt.tight_layout()

nn_importance_path = os.path.join(FIGURE_DIR, 'neural_network_permutation_importance.png')
plt.savefig(nn_importance_path, dpi=300, bbox_inches='tight')
plt.show()

print('Plot permutation importance Neural Network disimpan di:', nn_importance_path)


# Cell 64
# 12.1 Ringkasan performa akhir
```

```
final_summary = eval_df[
    ['Model', 'Accuracy', 'Sensitivity_Recall_CKD', 'Specificity_Not_CKD', 'AUC', 'Brier_score']
].copy()

display(final_summary)
```